

The Theory of NP-Completeness

5.1 Fundamental Considerations

With the help of reduction concepts like $\leq_T$ and $\leq_P$, we have been able to establish the algorithmic similarity of a number of problems. To this point, however, we have been proceeding rather unsystematically, and have only been becoming familiar with these notions of reduction. Now we want to investigate what we accomplish with further reductions between important problems.

Turing equivalence ($\equiv_T$) is an equivalence relation on the set of all algorithmic problems, and so partitions this set into equivalence classes. On the set of equivalence classes, Turing reducibility ($\leq_T$) leads to a partial order if for any two equivalence classes $\mathcal{C}_1$ and $\mathcal{C}_2$ we define $\mathcal{C}_1 \leq_T \mathcal{C}_2$ if $A \leq_T B$ holds for all $A \in \mathcal{C}_1$ and all $B \in \mathcal{C}_2$. This is equivalent to $A \leq_T B$ for some $A \in \mathcal{C}_1$ and some $B \in \mathcal{C}_2$. Clearly P forms one of these equivalence classes. For $A, B \in P$ we have $A \leq_T B$, since we don't even need to use an algorithm for B in order to compute A in polynomial time. On the other hand, if $B \in P$ and $A \leq_T B$, then by definition $A \in P$.

So if we get to know the partial order described by " $\leq_T$ " on these classes, then we will have learned much about the complexity of all problems. We are, however, still far from such a complete picture. Based on the results of Chapter 4, we can hope to show that many problems belong to the same equivalence class with respect to $\equiv_T$. It follows that either all of these problems are efficiently solvable or none of them is. This is the formalization of the comment made in Chapter 1 that out of a thousand secrets, one great overarching secret is formed. The previous comment can be extended to the complexity class ZPP and to the class of problems contained in BPP that have unique solutions or are optimization problems for which the value of a solution can be computed in polynomial time. This follows from the results obtained in Chapter 3 that show that in these cases we can reduce the error- or failure-probability so that even polynomially many calls have a sufficiently small error- or failure-probability. If we use BPP to denote this restricted BPP -class as well, we obtain the following:

For any set of Turing equivalent problems and each of the complexity classes P , ZPP , and BPP , it is the case that either all the problems belong to the complexity class or none of them do.

If we restrict our attention to decision problems, then polynomial reducibility ($\leq_P$) is also available. Recall that for the relation $A \leq_P B$ to hold, on each instance of problem A the algorithm for B may only be called once, but must be called. Furthermore, the answer to this call must provide the correct answer for our instance of problem A . The first of these restrictions simplifies our considerations somewhat since it is no longer necessary to reduce the error- or failure-probability.

For any set of polynomially equivalent decision problems and each of the complexity classes P , ZPP , $NP \cap \text{co-NP}$, RP , co-RP , NP , co-NP , BPP , and PP , it is the case that either all the problems belong to the complexity class or none of them do.

Because of the second condition on polynomial reductions, not all problems in P are polynomially equivalent. The class of decision problems that are solvable in polynomial time is subdivided into three categories:

- all problems for which no inputs are accepted,
- all problems for which every input is accepted,
- all other problems.

The first two equivalence classes offer no help as oracle, since they answer every query the same way, and for polynomial reductions this value cannot be changed. For all other problems A there is an input x that is accepted and an input y that is rejected. For a decision problem $B \in P$ we can show that $B \leq_P A$ as follows: On input z , we determine in polynomial time if $z \in B$. If $z \in B$, then we query the oracle A about x , otherwise we query about y .

Now we return to our main theme. We have a set of problems, among them some important problems, and we know that they are all Turing equivalent (or perhaps even polynomially equivalent), and we don't know a polynomial-time algorithm for any of them. Our previous observations lead to the following argument: We have so many different problems on which many computer scientists have worked hard for a long time. No one has found a polynomial-time algorithm for any of these problems. Based on the highly refined methods for the design and analysis of algorithms, we arrive at the suspicion that none of these problems is solvable in polynomial time. For each problem, this conjecture is supported by the failed attempts to solve all the other problems.

The theory of NP-completeness is the successful attempt to structurally support the conjectured algorithmic difficulty of classes of problems in view of the complexity classes considered in Chapter 3. In Chapter 4 we saw that it suffices to consider decision problems. The important optimization problems are Turing equivalent to their decision variants. So we are dealing with the

complexity landscape described in Theorem 3.5.3. With the help of the following definition, we compare the complexity of a problem A with the most difficult problems in a complexity class $\mathcal{C}$.

Definition 5.1.1. *Let A be a decision problem and $\mathcal{C}$ a class of decision problems.*

- i) A is $\mathcal{C}$ -hard with respect to polynomial reductions ($\leq_p$ -hard for $\mathcal{C}$), if $C \leq_p A$ for every $C \in \mathcal{C}$.
- ii) A is $\mathcal{C}$ -easy with respect to polynomial reductions ($\leq_p$ -easy for $\mathcal{C}$) if $A \leq_p C$ for some $C \in \mathcal{C}$.
- iii) A is $\mathcal{C}$ -equivalent with respect to polynomial reductions ($\leq_p$ -equivalent for $\mathcal{C}$) if A is both $\mathcal{C}$ -hard and $\mathcal{C}$ -easy with respect to polynomial reductions.
- iv) A is $\mathcal{C}$ -complete with respect to polynomial reductions ($\leq_p$ -complete for $\mathcal{C}$), if A is $\leq_p$ -hard for $\mathcal{C}$ and $A \in \mathcal{C}$.

The term “ $\mathcal{C}$ -hard” signals that A is at least as hard as every problem in $\mathcal{C}$. In particular, A does not belong to P if $\mathcal{C}$ contains any problems not in P . The counterpart of $\mathcal{C}$ -hard is $\mathcal{C}$ -easy. If A is $\mathcal{C}$ -easy and $\mathcal{C}$ contains only efficiently solvable problems, then A is also efficiently solvable. By considering all $C \in \mathcal{C}$ in the definition of $\mathcal{C}$ -hard and only one $C \in \mathcal{C}$ in the definition of $\mathcal{C}$ -easy, we are implicitly comparing A to a “hardest problem” in $\mathcal{C}$. The complexity class $\mathcal{C}$ is thus represented (as far as its computational power is concerned) by its hardest problems. If A is $\mathcal{C}$ -equivalent, then $A \in P$ if and only if $\mathcal{C} \subseteq P$.

Since $\leq_p$ is only a partial order on the set of decision problems, a complexity class $\mathcal{C}$ can contain many “hardest problems” that are not comparable with respect to polynomial reductions. But if $\mathcal{C}$ contains a $\mathcal{C}$ -complete problem A , then the class of all $\mathcal{C}$ -complete problems forms an equivalence class with respect to $\equiv_p$ within the class $\mathcal{C}$. This means that the $\mathcal{C}$ -complete problems are all equally difficult and are the hardest among the problems in $\mathcal{C}$. If we suspect that $\mathcal{C}$ is not contained in P , then this is equivalent to the conjecture that none of the $\mathcal{C}$ -complete problems belongs to P . A proof of the $\mathcal{C}$ -hardness of a problem for a class $\mathcal{C}$ that is conjectured not to be contained in P is therefore a strong indication that this problem does not belong to P . Every $\mathcal{C}$ -complete problem is also $\mathcal{C}$ -equivalent, and by definition $\mathcal{C}$ -hard. Since $A \leq_p A$ and $A \in \mathcal{C}$, A is also $\mathcal{C}$ -easy.

In order to include optimization problems, and in fact all algorithmic problems, in our considerations, we extend our definitions to these classes of problems. For this we must use Turing reductions in place of polynomial reductions.

Definition 5.1.2. *Let A be an algorithmic problem and let $\mathcal{C}$ be a class of algorithmic problems. Then the terms $\mathcal{C}$ -hard with respect to Turing reductions, $\mathcal{C}$ -easy with respect to Turing reductions, and $\mathcal{C}$ -equivalent with respect to Turing reductions are defined analogously to the terms in Definition 5.1.1 by replacing polynomial reductions ($\leq_p$) with Turing reductions ($\leq_T$).*

We follow the convention that $\mathcal{C}$ -hard, $\mathcal{C}$ -easy, and $\mathcal{C}$ -equivalent (without any mention of the type of reduction) will be understood to be with respect to Turing reductions. $\mathcal{C}$ -completeness, on the other hand, will be understood to be with respect to polynomial reductions. In fact, we did not even include completeness with respect to Turing reductions in Definition 5.1.2. A significant reason for this is the investigation of decision problems L and their complements $\bar{L}$. All the classes we are interested in are closed under polynomial reductions, but probably not all are closed under Turing reductions. In particular, it is possible that L be $\mathcal{C}$ -complete (with respect to polynomial reductions), but that $\bar{L} \notin \mathcal{C}$. Many interesting classes are not known to be closed under complement, and if a class is not closed under complement, then Turing-completeness, while still sensible, has the funny property that the set of all problems Turing-equivalent to a Turing-complete problem is not entirely contained in that class. It is worth noting that if L is $\mathcal{C}$ -complete then $\bar{L}$ is at least $\mathcal{C}$ -equivalent.

Our goal is to prove that the decision variants of many of our problems are complete for some class $\mathcal{C}$ that is conjectured not to be contained in P . If the optimization variant of a problem is Turing equivalent to its decision variant, then it will immediately follow that the optimization problem is $\mathcal{C}$ -equivalent.

There are only a few decision problems that we know belong to ZPP, RP, co-RP, or BPP, that we don't know to be contained in P . This already makes the study of, for example, BPP-completeness less attractive. Furthermore, we are currently not at all sure that $P \neq BPP$. The error-probability of BPP algorithms can be made exponentially small, and in many cases it has been possible to *derandomize* these algorithms, that is, to replace the randomized algorithm with a deterministic algorithm that has a runtime that is only polynomially greater than the original randomized algorithm. (See Miltersen (2001).) So BPP is generally considered to be “at most a little bit bigger than P ”. The situation looks very different for the classes $NP \cap co-NP$, NP , $co-NP$, and PP . No one believes that such large error- or failure-probabilities can be derandomized. In Section 5.2 we will show that the decision variants of the important optimization problems are contained in NP . Since we don't believe them to be in $NP \cap co-NP$, it makes sense to begin an investigation of the complexity class NP .

As we have already discussed, the complexity classes $NP \cap co-NP$, NP , $co-NP$, and PP have no direct algorithmic relevance, but they form the basis for the complexity theoretical classification of problems. The decision variants of thousands of problems have been shown to be NP -complete, and their optimization variants to be NP -equivalent. So we are in the following situation:

Thousands of important problems are NP -complete or NP -equivalent. Either all of these problems can be solved in polynomial time and $NP = P$, or none of these problems can be solved in polynomial time and $NP \neq P$.

The $\text{NP} \neq \text{P}$ -problem offers a central challenge for complexity theory and indeed for all of theoretical computer science. The Clay Mathematical Institute has included this problem in its list of the seven most important problems connected to mathematics and offered a reward of \$1,000,000 for its solution.

We do not know if $\text{NP} = \text{P}$ or if $\text{NP} \neq \text{P}$. Since all experts believe with good reason that $\text{NP} \neq \text{P}$, a proof that a problem is NP -complete, NP -equivalent, or NP -hard is a strong indication that the problem is not solvable in polynomial time.

5.2 Problems in NP

We want to show that the decision variants of all the problems introduced in Section 2.2 belong to NP . To do this we must design NP algorithms, i.e., randomized algorithms with one-sided error where the error-probability must only be less than 1. Said differently: The probability that x is accepted is positive if and only if $x \in L$. These algorithms have no practical relevance, they only serve to show that the problems considered belong to the class NP .

Theorem 5.2.1. *The decision variants of all the problems introduced in Section 2.2 belong to the class NP .*

Proof. In this proof we will see that NP algorithms for many problems follow the same outline. For an input of length n , let $l(n)$ be the length of a potential solution. Then the following steps are carried out:

- The first $l(n)$ random bits are stored.
- These $l(n)$ bits are tested to see if they describe a potential solution. If not, the input is rejected.
- In the positive case, the solution is checked to see if it has the required quality, and if so the input is accepted.

Clearly every input that should not be accepted will be rejected with probability 1. If there is a solution with the required quality, then this solution will be randomly drawn with probability at least $2^{-l(n)} > 0$, so in this case the input is accepted. The computation time is polynomially bounded if $l(n)$ is polynomially bounded and testing whether a sequence of bits represents a solution and, if so, determining its quality can both be done in polynomial time. This is easy to show for the problems we are considering (and for many others as well).

Traveling Salesperson Problems (TSP): A tour is a sequence of n cities $i_1, \dots, i_n \in \{1, \dots, n\}$ and can be represented with $n \lceil \log(n+1) \rceil$ bits. It is possible to check in polynomial time whether $\{i_1, \dots, i_n\} = \{1, \dots, n\}$ (i.e., whether we have a tour at all), and if so it is possible in polynomial time to compute the value of the circuit.

Knapsack Problems: Each $a \in \{0, 1\}^n$ represents a selection of objects. In linear time we can decide if this selection abides by the weight limit and, if so, the utility can also be computed in linear time.

Scheduling Problems (BINPACKING): We let $l(n) = n \lceil \log n \rceil$ and interpret the i th block of length $\lceil \log n \rceil$ as the number of the bin in which we place the i th object. We then check that no bin has been overfilled and finally determine the number of bins used.

Covering Problems (VERTEXCOVER, EDGECOVER): Each $a \in \{0, 1\}^n$ or $\{0, 1\}^m$ represents a subset of the vertices or edges, respectively. It is then easy to check if all edges or vertices are covered and to compute the number of vertices or edges selected.

Clique Problems (CLIQUE, INDEPENDENTSET, CLIQUECOVER): For CLIQUE and INDEPENDENTSET, the procedure is similar to that for VERTEXCOVER, this time testing if the selected set of vertices is a clique or anti-clique. For CLIQUECOVER the vertices are partitioned as was the case for BINPACKING. Each set in the partition is checked to see if it is a clique.

Team Building Problems (k -DM): Again, as for BINPACKING, a partition into teams is made and tested to see if each team contains exactly one member from each of the k groups of people.

Network Flow Problems (NETWORKFLOW): The decision variant is in P and, therefore, in NP.

Championship Problems (CHAMPIONSHIP): If there are m remaining games and r possible point distributions per game, then $m \lceil \log r \rceil$ bits suffice to describe any combination of outcomes. Once the outcomes of the games are given, it is easy to decide if the selected team is league champion.

Verification problems (SAT): Each $a \in \{0, 1\}^n$ describes a truth assignment. Even for circuits it is easy to test if a given truth assignment satisfies the formula.

Number Theory Problems (PRIMES): it is easy to show that PRIMES $\in$ co-NP. Each potential divisor $j \in \{2, \dots, n-1\}$ has a binary representation of length at most $\lceil \log n \rceil$. We can efficiently decide if n/j is an integer. The randomized primality test of Solovay and Strassen even shows that PRIMES $\in$ co-RP. In addition, it has been known for a long time that PRIMES $\in$ NP, although the proof of this fact is not as easy as the others we have presented here. More recently Agrawal, Kayal, and Saxena (2002) succeeded in showing that PRIMES $\in$ P (see also Dietzfelbinger (2004)). $\square$

The decision variants of the problems we have investigated (and of many others as well) belong to the class NP. The NP algorithms are very efficient but, due to their high error-probabilities, algorithmically worthless.

5.3 Alternative Characterizations of NP

For randomized algorithms with a runtime bounded by a polynomial $p(n)$, we want to separate the generation of random bits from the actual work of the algorithm. First, $p(n)$ random bits are generated and stored. Then A is “deterministically simulated” by using these stored random bits instead of generating random bits as the algorithm goes along. Aside from the time needed to compute $p(n)$, this gives rise to doubling of the worst-case runtime. That is,

When considering randomized algorithms, we can restrict our attention to algorithms that work in two phases:

- *Phase 1: determination of the input length n , computation of $p(n)$ (for a polynomial p), generation and storage of $p(n)$ random bits;*
- *Phase 2: a deterministic computation that requires at most $p(n)$ steps and uses one random bit per step.*

In the imaginary world of nondeterministic algorithms, in the first phase sufficiently many bits are “guessed” (nondeterministically generated), and in the second phase they are tested to see if the guessed bits “verify” that the input can be accepted. One refers to this as the method of “guess and verify”. These considerations do not lead to better algorithms, but they simplify the structural investigation of problems in NP, as we will show in two examples.

Theorem 5.3.1. *Every decision problem in NP can be solved by a deterministic algorithm with runtime bounded by $2^{q(n)}$ for some polynomial q .*

Proof. Let $p(n)$ be a polynomial that bounds the runtime of an NP algorithm A for a particular problem. The random bits generated in phase 1 of the algorithm form a random sequence from $\{0, 1\}^{p(n)}$. It is easy to step through the $2^{p(n)}$ possible random bit sequences and to deterministically simulate the NP algorithm on each one in turn. The total computation time is bounded by $O(p(n)2^{p(n)})$, which can be bounded by $2^{q(n)}$ for some polynomial $q(n) = O(p(n))$. The deterministic algorithm accepts the input if and only if A accepts on at least one of the random sequences. By the definition of NP, the deterministic algorithm always makes the correct decision. $\square$

We can also interpret the two phases of an NP algorithm as follows: An input x of length n is extended by a 0-1 vector z of length $p(n)$. Then the deterministic algorithm A' of phase 2 works on the input (x, z) . This leads to the following characterization of NP.

Theorem 5.3.2. *A decision problem L is in NP if and only if there is a decision problem $L' \in \mathbf{P}$ such that L can be represented as*

$$L = \left\{ x \mid \exists z \in \{0, 1\}^{p(|x|)} : (x, z) \in L' \right\} .$$

Proof. If $L \in \text{NP}$, then we can implement the discussion preceding the statement of the theorem. Assume we have a 2-phase algorithm A . The number of random bits on an input of length n is bounded by a polynomial $p(n)$, and the deterministic algorithm of phase 2 accepts a language $L' \in P$. An input x belongs to L if and only if there is a setting z of the random bits that causes the deterministic algorithm to accept (x, z) in the second phase. This is precisely the desired characterization.

If L can be characterized as in the statement of the theorem, then a randomized algorithm on input x can generate a random bit sequence z of length $p(|x|)$ and then check deterministically if $(x, z) \in L'$. If $x \in L$, then it will be accepted with positive probability, but if $x \notin L$, then it will never be accepted. $\square$

The $\text{NP} \neq \text{P}$ -hypothesis can now be seen in a new light. Decision problems belong to NP if and only if they can be represented by a polynomially length-bounded existential quantifier ($\exists z \in \{0, 1\}^{p(|x|)}$) and a polynomially decidable predicate ($((x, z) \in L')$). The $\text{NP} \neq \text{P}$ -hypothesis is equivalent to the claim that the existential quantifier enlarges the set of represented problems. This characterization of NP is also known as the *logical* characterization of NP . Using DeMorgan's Laws we immediately obtain a logical characterization of co-NP as well, namely as the class of all languages L such that

$$L = \left\{ x \mid \forall z \in \{0, 1\}^{p(|x|)} : ((x, z) \in L') \right\}$$

for some polynomial p and some language $L' \in P$. The conjecture that we cannot replace the existential quantifier in this characterization with a universal quantifier is equivalent to the statement that $\text{NP} \neq \text{co-NP}$. The results from Section 5.2 can also be interpreted in this light. The decision variants of optimization problems are defined using existential quantifiers (e.g., is there a tour?) and polynomial predicates (is the cost bounded by D ?). On the other hand, the set of prime numbers n is defined using a universal quantifier: For all k with $2 \leq k < n$, k is not a divisor of n .

5.4 Cook's Theorem

Now we come to the seminal result that made the theory of NP -completeness for particular problems possible. We have seen through many examples that there are polynomial reductions between very different looking problems. Many decision variants of important optimization problems belong to NP , and the complexity class NP can be characterized not only in terms of algorithms, but also logically. But none of this brings us any closer to a proof that any particular problem is NP -complete, or even that there are any NP -complete problems. The hurdle lies in the definition of NP -completeness. To show that a problem is NP -complete, we need to reduce *every* problem in NP to our

selected problem. That is, we must argue about problems about which we know nothing other than the fact that they belong to NP. Cook (1971) and independently Levin (1973) were able to clear this hurdle.

Here too we want to proceed algorithmically and keep the necessary resources as small as possible. Let L be a decision problem in NP for which there is an algorithm with runtime bounded by a polynomial $p(n)$. We have seen that the runtimes for most of the problems we are interested in (assuming a random access machine model) have been linear or at least quasi-linear. If we use Turing machines instead, the computation time is roughly squared, although for Turing machines with multiple tapes the slowdown can often be greatly reduced. Turing machines prove useful in this context because every step has only local effects. For every step t in a computation of a $q(n)$ -bounded Turing machine, we can take an instantaneous snapshot of the Turing machine, which we will call the configuration at time t . It consists of the current state, the current contents of the tape cells, and the current location of the tape head. So inputs $x \in L$ can be characterized as follows: There are configurations $K_0, K_1, \dots, K_{q(n)}$ such that K_0 is the initial configuration on input x , $K_{q(n)}$ is an accepting configuration (i.e., the current state is an accepting state $q \in Q^+$), and K_i , $1 \leq i \leq q(n)$, is a legal successor configuration of K_{i-1} . K_i is a legal successor configuration of K_{i-1} means that the Turing machine, for one of the two choices of the random bit used at step i of the computation, transforms configuration K_{i-1} into K_i .

Which of the problems has such properties that we hope will simplify our work of creating a transformation from the conditions just described into instances of that problem? Whether q is the current state at time t , or whether the tape head is at tape cell j at time t , etc., can be easily described using Boolean variables, and we are interested in whether certain dependencies between these variables are *satisfied*. So we have arrived at verification problems, and we select SAT as our target problem.

Although Turing machines work locally, for the i th configuration there are $2i + 1$ possible tape head positions. In order to simplify the arguments that follow, we will first simulate Turing machines with machines that work “even more locally”.

Definition 5.4.1. *A Turing machine is oblivious if the position of the tape head until the time the machine halts depends only on the step t and not on the input x .*

Lemma 5.4.2. *Every (deterministic or randomized) Turing machine M can be simulated by a (deterministic or randomized, respectively) oblivious Turing machine M' in such a way that t computation steps of M are simulated by $O(t^2)$ computation steps of M' .*

Proof. The sequence of positions of the tape head of the Turing machine M' will be the following regardless of the Turing machine being simulated:

$$\begin{aligned}
&0, 1, 0, -1, \\
&-1, 0, 1, 2, 1, 0, -1, -2, \\
&-2, -1, 0, 1, 2, 3, 2, 1, 0, -1, -2, -3, \dots, \\
&-(j-1), \dots, 0, \dots, j, \dots, 0, \dots, -j, \dots
\end{aligned}$$

The j th phase of the computation of M' will consist of $4j$ computation steps and will simulate the j th step of the Turing machine M . Thus t computation steps of M are simulated with

$$4 \cdot (1 + 2 + \dots + t) = 2t(t+1) = O(t^2)$$

steps of M' .

How does the simulating Turing machine M' work? How does it count the positions? All of this is solved with a few simple tricks and by increasing the size of the tape alphabet and the internal memory (number of states). Prior to the t th step of M , cells $-(t-1)$ and $t-1$ should be marked to indicate the left and right ends of the portion of the tape being considered. This is not possible for $t=1$, but for the first step the Turing machine can call upon its internal memory and “imagine” the two marks, both of which would mark cell 0 at the start of the computation. In addition, we must mark the tape position that machine M reads at step t . For step $t=1$ this is again cell 0 and this information is again stored in the internal memory. The internal memory also keeps track of the state of machine M at each step, starting with the initial state of M for $t=1$. The simulation is now easy to describe.

Starting from position $-(t-1)$, M' moves to the right looking for the cell that M reads at step t . This is easy to find with the help of its special marking. At this point, M' knows all the information that M has during its t th step. The state q is stored in the internal memory, and the symbol a that M reads from the tape, M' can also read from the tape (in the same cell that contains the tape head marker). If necessary, M' uses a random bit just like M . Now M' can store the new state q' in its memory and write the new tape symbol a' to the tape cell. Furthermore, M' knows if the tape head marker must be moved, and if so in which direction. For a move to the right, this is done in the next step. For a move to the left, M' remembers that this is still to be done. M' then searches for the right end marker and moves it to the right one cell. Then M' moves back to the left end marker and moves it one space to the left. If necessary, the tape head marker is moved to the left along the way. At this point, M' is ready to simulate the $(t+1)$ st step of M . If M halts, then M' halts as well, making the same decision. $\square$

For our purposes, this simulation is fully adequate. If we use Turing machines with k tapes, we can use the same ideas to simulate them with oblivious one-tape Turing machines (see, for example, Wegener (1987)). Now we are prepared for Cook's Theorem.

Theorem 5.4.3 (Cook's Theorem). *SAT is NP-complete. Thus $\text{NP} = \text{P}$ if and only if $\text{SAT} \in \text{P}$.*

Proof. Since SAT is in NP, it is sufficient to show that every decision problem $L \in \text{NP}$ can be polynomially reduced to SAT. By Lemma 5.4.2, we can assume that the NP algorithm for L has a worst-case runtime of $p(n)$ and is implemented on an oblivious one-tape Turing machine. The reduction is a transformation with connected components. The components of the Turing machine M on input x with $|x| = n$ consist of the states, the random bits, and the contents of the tape. Here we use the fact that the tape-head position is independent of x . The components are represented by Boolean variables. To simplify notation we assume that the states are $q_0, \dots, q_{k-1}$, that the tape alphabet consists of the symbols $a_1, \dots, a_m$, and that a_m is the blank symbol.

- $Q(i, t)$, $0 \leq i \leq k-1$, $0 \leq t \leq p(n)$: $Q(i, t) = 1$ represents that at step t , M is in state q_i . (The 0th step corresponds to the initialization.)
- $Z(t)$, $1 \leq t \leq p(n)$, represents the value of the t th random bit.
- $S(i, t)$, $1 \leq i \leq m$, $0 \leq t \leq p(n)$: $S(i, t) = 1$ represents that at step t , the symbol a_i is being read from the tape.

All together we use $(p(n) + 1) \cdot (|Q| + 1 + |\Gamma|) - 1 = O(p(n))$ Boolean variables.

The clauses will express the way M works in such a way that there will be values of the random bits $Z(1), \dots, Z(p(n))$ for which M accepts input x if and only if the clauses are simultaneously satisfiable. Furthermore, we need to ensure that the variables represent things the way we are imagining. That is, we must satisfy the following conditions.

1. The variables for $t = 0$ correspond to the initial configuration of the computation.
2. The last configuration is accepting.
3. The variables represent configurations.
4. The t th configuration is the successor configuration of the $(t-1)$ st configuration according to the instructions of the Turing machine M .

We code these conditions as a conjunction of clauses. This can be done separately for each condition since conditions (1)–(4) are conjunctively joined.

(1) Since M starts in state q_0 , it must be the case that $Q(0, 0) = 1$ and $Q(i, 0) = 0$ for all $i \neq 0$. For each j , let $t(j)$ be the first time that cell j is read. For each $0 \leq j \leq n-1$, $S(i, t(j))$ must have the value 1 if and only if $x_{j+1} = a_i$. (This is the only point at which the input x influences the transformation.) For all other j , $S(m, t(j))$ must have the value 1, since at the beginning of the computation the corresponding cells contain the blank symbol. Here we have not really made any clauses, rather we have replaced some of the Boolean variables with appropriate Boolean constants.

(2) For this we need a clause that is the disjunction of all $Q(i, p(n))$ for which q_i is an accepting state.

(3) At every time, the Turing machine is in exactly one state and reads exactly one symbol. Syntactically, this is the only condition placed on configurations. Formally, this means that for every $t \in \{0, \dots, p(n)\}$, exactly one of

the variables $Q(i, t)$ and exactly one of the variables $S(i, t)$ has the value 1. So we have $2p(n) + 2$ conditions, of which some have already been fulfilled by our actions under (1). Since the number of variables in each condition is $|Q|$ or $|I|$, and therefore $O(1)$, we can make our lives easy. Every Boolean function can be represented in conjunctive normal form, that is, as a conjunction of disjunctions (clauses). The number of clauses with r variables is bounded by 2^r , which in our case is $O(1)$, since the number of variables is $O(1)$. (In fact, for our function “exactly one input variable has the value 1”, $O(r^2)$ clauses suffice.)

(4) Now we must code the semantics of the Turing machine M . The t th step of M depends on the state after the $(t - 1)$ st step (i.e., $Q(i, t - 1)$, $0 \leq i \leq k - 1$), on the random bit $Z(t)$, and on the symbol being read at time t (i.e., $S(i, t)$, $1 \leq i \leq m$). This is $|Q| + |I| + 1 = O(1)$ variables. The result of this step in the computation is expressed in the new state (i.e., the variables $Q(i, t)$ for $0 \leq i \leq k - 1$) and in the symbol that is written to the tape cell (i.e., $S(i, N(t))$ for $1 \leq i \leq m$, where $N(t)$ is the next time after time t that M once again reads the cell that is read at time t). If $N(t) > p(n)$, then this information is irrelevant and need not be computed. In other words, the following $|Q| + |I| = O(1)$ equations must be satisfied for $0 \leq i \leq k - 1$ and $1 \leq j \leq m$ to guarantee that M is being simulated:

$$Q(i, t) = f_i(Q(0, t - 1), \dots, Q(k - 1, t - 1), Z(t), S(1, t), \dots, S(m, t))$$

and

$$S(j, N(t)) = g_j(Q(0, t - 1), \dots, Q(k - 1, t - 1), Z(t), S(1, t), \dots, S(m, t)) .$$

These $|Q| + |I|$ equations describe δ in our coding of states, symbols, and random bits. So these functions don't depend on t either. Every equation is true only for certain assignments of the variables that occur, and can therefore be expressed as a conjunction of $O(1)$ clauses. (An explicit description of the clauses can be found in Garey and Johnson (1979), but this is not needed for our purposes.)

All together we have $O(p(n))$ clauses of length $O(1)$, i.e., an instance of SAT with total length $O(p(n))$. The clauses can be computed in time $O(p(n))$. By first simulating all the movements of the Turing machine M for $p(n)$ steps, we can compute $t(j)$ and $N(t)$ for all t in time $O(p(n))$. Since the individual functions and equations each contain only $O(1)$ variables, they can be converted to conjunctive normal form in time $O(1)$.

If M accepts input x with random bits $z_1, \dots, z_{p(n)}$, we can replace the variables in our SAT instance with the values that they represent in the computation of M and this will satisfy all the clauses. On the other hand, if there is an assignment for the variables that satisfies all of the clauses, then we obtain an accepting computation of M by setting the random bits according to the values of the variables $Z(t)$ in the satisfying assignment. Condition (1) ensures

that M is correctly initialized. Condition (3) ensures that the variables represent a current state and a read tape symbol at every step in the computation. Condition (4) ensures inductively that the states and tape symbols follow the computation of M . Finally, condition (2) ensures that the computation accepts its input. $\square$

Using this pioneering result it is significantly easier to prove the NP-completeness of additional decision problems. This is because of the transitivity of $\leq_p$. If $L \in \text{NP}$ and we can show that $K \leq_p L$ for some NP-complete problem K , then L is also NP-complete since for all $L' \in \text{NP}$, $L' \leq_p K$, and from $K \leq_p L$ it follows that $L' \leq_p L$ for all $L' \in \text{NP}$. Proofs of NP-completeness will tend to get easier and easier, since the number of known NP-complete problems is continually growing, and so we have an ever-growing selection of problems, and it is sufficient to polynomially reduce any one of them to a new problem to show that the new problem is NP-complete.

To prove the NP-completeness of a problem $A \in \text{NP}$ it is sufficient to reduce some NP-complete problem to A .

By the results from Chapter 4, we now know that the following problems are all NP-complete: SAT, 3-SAT, CLIQUE, INDEPENDENTSET, VERTEXCOVER, DHC, HC, and TSP^{2, Δ ,sym}. The optimization variants of these problems are all NP-equivalent.